Y — Retea de Socializare

Documentul de Proiectare Arhitecturala (ADD)

Architectural Design Document

Campul	Detalii
Versiune	1.0
Data	Aprilie 2026
Proiect	Y - Retea de Socializare (Y-Social-Media)
Repository	github.com/NaxonRO/Y-Social-Media
Status	In dezvoltare - Proiect scolar
Metodologie	Agile/Scrum - 4 Sprint-uri x 2 saptamani

Cuprins

1. Introducere ... 3
1.1 Scopul sistemului ... 3
1.3 Definitii si Acronime ... 3
1.4 Documente de referinta ... 4
2. Obiective de Proiectare ... 4
3. Arhitectura Propusa ... 5
3.1 Decompozitia in Subsisteme ... 5
3.1.1 Subsistemul AUTH ... 5
3.1.2 Subsistemul CONTENT ... 6
3.1.3 Subsistemul SOCIAL ... 6
3.1.4 Subsistemul MESSAGING ... 7
3.1.5 Subsistemul NOTIFICATIONS ... 7
3.2 Distributia pe Platforme Hardware/Software ... 8
3.3 Managementul Datelor Persistente ... 9
3.3.1 Schema conceptuala a bazei de date ... 9
3.3.2 Strategia de cache si date temporare (Redis) ... 10
3.4 Controlul Accesului Utilizatorilor ... 10
3.5 Fluxul Global al Controlului ... 11
3.6 Tratarea Conditiilor Limita ... 12
3.6.1 Pornirea sistemului ... 12
3.6.2 Oprirea sistemului ... 12
3.6.3 Tratarea erorilor si conditii speciale ... 12
3.6.4 Cazuri de utilizare administrative ... 13
4. Planul de Implementare — 4 Sprint-uri ... 13
Sprint 1 — Implementare 1: Fundatia & Autentificarea ... 14
Sprint 2 — Implementare 2: Core Functionalities ... 15
Sprint 3 — Implementare 3: Social Features & Notificari ... 17
Sprint 4 — Finalizare: Polish, Testare & Deployment ... 19
5. Calendarul General al Proiectului ... 21
Glosar de Termeni ... 21

1. Introducere

1.1 Scopul sistemului

Y este o aplicatie mobila de micro-blogging si retea de socializare destinata utilizatorilor din Romania. Sistemul permite publicarea de postari scurte, interactiunea sociala (like, comentariu, repost, follow), mesagerie directa, notificari in timp real si descoperirea continutului prin hashtag-uri.

Aplicatia este inspirata din platforme internationale (X/Twitter) dar adaptata culturii si limbii romane, cu accent pe comunitate locala, confidentialitate (GDPR) si performanta pe dispozitive mid-range.

1.3 Definitii si Acronime

Termen / Acronim	Definitie
Y	Denumirea aplicatiei mobile de socializare
ADD	Architectural Design Document - documentul curent
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token - mecanism de autentificare stateless
DM	Direct Message - mesaj privat intre doi utilizatori
Feed	Fluxul de postari afisat pe ecranul principal
Hashtag (#)	Eticheta textuala pentru categorizarea continutului
Follow	Actiunea de a subscrie la postarile unui utilizator
Repost	Redistribuirea postarii altui utilizator
CDN	Content Delivery Network - retea de livrare rapida a continutului media
GDPR	General Data Protection Regulation - regulamentul european privind datele personale
FCM	Firebase Cloud Messaging - serviciu de notificari push pentru Android
WebSocket	Protocol de comunicare bidirectionala in timp real
UC	Use Case - Caz de utilizare

1.4 Documente de referinta

- Documentul de Specificare a Cerintelor (SRS) - Y Social Media v1.0, Martie 2026
- README.md - github.com/NaxonRO/Y-Social-Media
- CONTRIBUTING.md - Ghid de contributie la proiect
- Documentul de Proiectare Arhitecturala (ADD) - template furnizat de institutia academica
- React Native Documentation - reactnative.dev
- Node.js / Express.js Documentation
- PostgreSQL 15 Documentation
- RFC 7519 - JSON Web Token (JWT)
- Regulamentul (UE) 2016/679 - GDPR

2. Obiective de Proiectare

Arhitectura sistemului Y a fost conceputa pornind de la urmatoarele obiective principale:

ID	Obiectiv	Prioritate	Sprint tinta	Masura de succes
OB-01	Arhitectura modulara client-server cu separarea clara a responsabilitatilor (MVC)	Critica	Sprint 1	Structura de cod validata prin code review
OB-02	Autentificare securizata cu JWT si hashing parole (bcrypt)	Critica	Sprint 1	Zero vulnerabilitati de autentificare in audit
OB-03	API REST cu documentatie completa (Swagger/Postman) pentru toate endpoint-urile	Ridicata	Sprint 2	100% endpoint-uri documentate
OB-04	Timp de raspuns sub 2 secunde pentru operatii principale in conditii normale de retea	Ridicata	Sprint 3	Load test: p95 < 2s la 1000 req/s
OB-05	Scalabilitate orizontala a back-end-ului pentru minimum 10.000 utilizatori concurenti	Ridicata	Sprint 4	Test de stres cu 10k conexiuni simultane
OB-06	Comunicare in timp real prin WebSocket pentru mesaje si notificari	Medie	Sprint 3	Latenta < 500ms pentru livrare mesaje
OB-07	Conformitate GDPR: export date, drept la uitare, stocare in UE	Medie	Sprint 4	Audit GDPR trecut complet
OB-08	Disponibilitate sistem 99.5% uptime lunar (cloud-native, auto-scaling)	Medie	Sprint 4	Monitorizare cu alerta la downtime

3. Arhitectura Propusa

3.1 Decompozitia in Subsisteme

Sistemul Y este structurat in 5 subsisteme principale, fiecare cu responsabilitati bine definite si interfete clare:

3.1.1 Subsistemul de Autentificare si Gestionare Conturi (AUTH)

Atribut	Detalii
Functionalitati principale	Inregistrare, autentificare, logout, recuperare parola, gestionare sesiuni JWT, OAuth 2.0
Componente	AuthController, UserService, JWTService, EmailService, PasswordHasher (bcrypt)
Interfete expuse	POST /auth/register, POST /auth/login, POST /auth/logout, POST /auth/refresh, POST /auth/forgot-password
Interfete consumate	Baza de date PostgreSQL (tabela users, sessions), Redis (blacklist tokenuri)
Tehnologii	Node.js/Express, bcrypt, jsonwebtoken, Nodemailer, Redis

3.1.2 Subsistemul de Continut si Feed (CONTENT)

Atribut	Detalii
Functionalitati principale	Creare/editare/stergere postari, upload media, generare feed personalizat, like/comment/repost
Componente	PostController, FeedService, MediaUploadService, LikeService, CommentService, RepostService
Interfete expuse	GET /feed, POST /posts, DELETE /posts/:id, POST /posts/:id/like, POST /posts/:id/comment, GET /posts/:id
Interfete consumate	PostgreSQL (posts, comments, likes, reposts), AWS S3 (media), Redis (cache feed)
Tehnologii	Node.js/Express, Multer (upload), Sharp (procesare imagini), AWS SDK

3.1.3 Subsistemul Social (SOCIAL)

Atribut	Detalii
Functionalitati principale	Follow/unfollow, profiluri utilizatori, cautare, hashtag-uri, trending, grupuri
Componente	ProfileController, FollowService, SearchService, HashtagService, TrendingService
Interfete expuse	GET /users/:id, POST /users/:id/follow, DELETE /users/:id/follow, GET /search, GET /hashtags/:tag/posts
Interfete consumate	PostgreSQL (users, followers, hashtags, post_hashtags), Redis (cache cautare, trending)
Tehnologii	Node.js/Express, PostgreSQL full-text search, Redis Sorted Sets (trending)

3.1.4 Subsistemul de Mesagerie (MESSAGING)

Atribut	Detalii
Functionalitati principale	Conversatii private (DM), mesaje in timp real, status citire, cautare in mesaje
Componente	MessageController, ConversationService, WebSocketGateway, ReadReceiptService
Interfete expuse	GET /conversations, POST /conversations/:id/messages, WebSocket: ws://api/messaging
Interfete consumate	PostgreSQL (conversations, messages, read_receipts), Redis (pub/sub pentru real-time)
Tehnologii	Socket.io, Node.js, Redis Pub/Sub, PostgreSQL

3.1.5 Subsistemul de Notificari (NOTIFICATIONS)

Atribut	Detalii
Functionalitati principale	Generare notificari la evenimente, notificari push (FCM/APNs), centru de notificari in app
Componente	NotificationController, NotificationService, PushNotificationService, EventListener
Interfete expuse	GET /notifications, PATCH /notifications/:id/read, DELETE /notifications, WebSocket: ws://api/notifications
Interfete consumate	PostgreSQL (notifications), FCM (Android push), APNs (iOS push), Redis Pub/Sub
Tehnologii	Firebase Admin SDK, node-apn, Socket.io, Redis

3.2 Distributia Subsistemelor pe Platforme Hardware/Software

Diagrama de distributie a sistemului Y prezinta componentele fizice si logice distribuite pe infrastructura cloud:

Strat / Nod	Componente gazduite	Tehnologie / Provider
Client iOS (iPhone 8+, iOS 14+)	App mobila React Native, UI/UX, stare locala	React Native, Expo, iOS 14+
Client Android (Android 8+, 2GB+ RAM)	App mobila React Native, UI/UX, stare locala	React Native, Expo, Android 8+
API Gateway (Cloud)	Rate limiting, autentificare la nivel de retea, routing	AWS API Gateway / Nginx
Application Server (Cloud)	AUTH, CONTENT, SOCIAL, MESSAGING, NOTIFICATIONS subsystems	Node.js + Express, AWS EC2 / ECS
WebSocket Server (Cloud)	Real-time messaging, notificari live	Socket.io, AWS EC2 (sticky sessions)
Baza de date primara (Cloud)	Toate datele persistente: utilizatori, postari, mesaje, notificari	PostgreSQL 15, AWS RDS (UE)
Cache / Sesiuni (Cloud)	Cache feed, sesiuni, blacklist JWT, pub/sub real-time, trending	Redis 7, AWS ElastiCache
Stocare media (Cloud)	Imagini profil, poze postari, GIF-uri	AWS S3 + CloudFront CDN (UE)
Email Service (Extern)	Emailuri confirmare cont, recuperare parola	SendGrid / AWS SES
Push Notifications (Extern)	Notificari push Android si iOS	Firebase Cloud Messaging, APNs
CI/CD Pipeline (Cloud)	Build automat, testare, deploy	GitHub Actions, Docker, AWS ECR

3.3 Managementul Datelor Persistente

3.3.1 Schema conceptuala a bazei de date

Sistemul utilizeaza PostgreSQL 15 ca baza de date relationala principala. Schema urmatoare prezinta tabelele principale si relatiile dintre ele:

Tabela	Campuri principale	Relatii
users	id (PK, UUID), username (UNIQUE), email (UNIQUE), password_hash, display_name, bio, avatar_url, is_private, is_verified, created_at, updated_at	1-N: posts, notifications, conversations; N-M: followers (self-referential)
posts	id (PK, UUID), user_id (FK), content (max 280 chars), media_url, media_type, like_count, comment_count, repost_count, parent_id (FK, NULL), created_at	N-1: users; 1-N: comments, likes, reposts; N-M: hashtags
followers	follower_id (FK -> users), following_id (FK -> users), status (active/pending), created_at; PK: (follower_id, following_id)	Self-referential N-M pe tabela users
likes	user_id (FK), post_id (FK), created_at; PK: (user_id, post_id)	N-1: users; N-1: posts
comments	id (PK, UUID), user_id (FK), post_id (FK), content, parent_comment_id (FK, NULL), like_count, created_at	N-1: users, posts; self-referential pentru reply-uri
hashtags	id (PK), name (UNIQUE, lowercase), post_count, updated_at	N-M cu posts prin post_hashtags
post_hashtags	post_id (FK), hashtag_id (FK); PK: (post_id, hashtag_id)	Tabela de legatura posts-hashtags
conversations	id (PK, UUID), participant1_id (FK), participant2_id (FK), last_message_at, created_at	N-1: users (x2); 1-N: messages
messages	id (PK, UUID), conversation_id (FK), sender_id (FK), content, is_read, read_at, created_at	N-1: conversations, users
notifications	id (PK, UUID), recipient_id (FK), actor_id (FK), type (like/comment/repost/follow), post_id (FK, NULL), is_read, created_at	N-1: users (x2); N-1: posts (optional)
reposts	user_id (FK), post_id (FK), created_at; PK: (user_id, post_id)	N-1: users; N-1: posts

3.3.2 Strategia de cache si date temporare (Redis)

Tip date	Cheia Redis	TTL / Strategie
Feed utilizator	feed:{user_id}:page:{n}	TTL: 5 minute; invalidat la postare noua a utilizatorilor urmäriti
Sesiuni JWT (blacklist)	blacklist:jwt:{jti}	TTL: egal cu expirarea tokenului; set la logout
Trending hashtag-uri	trending:hashtags (Sorted Set)	Actualizat la fiecare postare; recalculat la 1 ora
Profil utilizator	user:profile:{user_id}	TTL: 15 minute; invalidat la editarea profilului
Real-time pub/sub	channel:user:{user_id}	Fara TTL; canal pub/sub pentru mesaje si notificari live

3.4 Controlul Accesului Utilizatorilor la Sistem

Sistemul Y implementeaza un model Role-Based Access Control (RBAC) cu 4 roluri distincte:

Rol	Vizualizare continut public	Gestionare cont propriu	Publicare continut	Moderare	Administrare sistem
Guest (vizitator)	DA	NU	NU	NU	NU
User (utilizator autentificat)	DA	DA	DA	NU	NU
Moderator	DA	DA	DA	DA	PARTIAL
Administrator	DA	DA	DA	DA	DA

Mecanismele de control al accesului:

- JWT (JSON Web Token): emis la autentificare, semnat cu cheia privata a serverului, contine rolul utilizatorului. Expira dupa 15 minute (access token) si 7 zile (refresh token).
- Middleware de autorizare: verificat pe fiecare endpoint protejat; extrage si valideaza JWT din header-ul Authorization: Bearer <token>.
- Protectia resurselor proprii: utilizatorul poate modifica/sterge doar propriile postari, comentarii, profil. Verificat prin compararea user_id din token cu owner_id al resursei.
- Conturi private: daca un utilizator are is_private = true, postările sale sunt vizibile doar urmăritorilor aprobati.

3.5 Fluxul Global al Controlului

Fluxul principal de control al sistemului urmeaza modelul REST API + WebSocket pentru operatiile in timp real:

Flux autentificare si acces resurse (REST)

Pas	Actor	Actiune	Componenta sistem
1	Utilizator	Trimite credentiale (email + parola)	POST /auth/login -> AuthController
2	AUTH Subsystem	Valideaza credentialele, genereaza JWT access + refresh token	JWTService, bcrypt.compare(), UserService
3	Client App	Stocheaza token-urile (Secure Storage), initiaza conexiune WebSocket	React Native SecureStore, Socket.io client
4	Utilizator	Acceseaza feed-ul principal	GET /feed -> FeedService -> PostgreSQL + Redis cache
5	Utilizator	Publica o postare noua	POST /posts -> PostController -> PostgreSQL + S3 (media) + Redis (invalidare cache feed)
6	NOTIFICATIONS Subsystem	Genereaza notificari pentru urmäritori	NotificationService -> PostgreSQL + FCM/APNs + Redis Pub/Sub
7	Utilizator	Trimite un mesaj direct	WebSocket emit -> MessageController -> PostgreSQL + Redis Pub/Sub -> WebSocket emit destinatar
8	Client App	Access token expirat; refresh automat	POST /auth/refresh -> JWTService -> nou access token

3.6 Tratarea Conditiilor Limita

3.6.1 Pornirea sistemului

- Ordinea de pornire: Redis -> PostgreSQL -> Application Server -> WebSocket Server
- Health check: fiecare serviciu expune GET /health; API Gateway verifica disponibilitatea inainte de a directiona trafic
- Migrari DB: rulate automat la startup prin sistem de migrare (ex: Flyway/Knex migrations)
- Preincarcarea cache-ului: trending hashtag-uri si profilurile populare sunt preincarcate in Redis la start

3.6.2 Oprirea sistemului (graceful shutdown)

- La primirea semnalului SIGTERM, serverul inceteaza sa accepte noi conexiuni
- Conexiunile WebSocket active sunt notificate cu mesajul de deconectare gratiosa
- Requesturile in curs sunt finalizate (timeout: 30 secunde)
- Conexiunile la baza de date si Redis sunt inchise ordonat

3.6.3 Tratarea erorilor si conditii speciale

Conditie	Comportament sistem	Mesaj catre utilizator
Token JWT expirat	Refresh automat din client; daca refresh token expirat -> logout fortat	"Sesiunea a expirat. Te rugam sa te autentifici din nou."
Postare > 280 caractere	Validare front-end si back-end; request respins cu HTTP 400	Contor caractere vizibil, buton blocat
Upload imagine > 5MB	Respins cu HTTP 413; imaginea nu este procesata	"Imaginea depaseste limita de 5MB."
Pierdere conexiune WebSocket	Reconectare automata cu exponential backoff (1s, 2s, 4s, max 30s)	Indicator "Reconectare..." in UI
Baza de date indisponibila	Citirile servite din cache Redis; scrierile puse in coada; alerta trimisa echipei	"Sistemul este temporar indisponibil."
Rate limiting depasit	HTTP 429 Too Many Requests; IP blocat temporar (sliding window)	"Prea multe requesturi. Asteptati {X} secunde."
Cont suspendat de admin	JWT invalidat imediat in Redis blacklist; redirect la pagina de suspendare	"Contul tau a fost suspendat. Contacteaza suportul."
Continut duplicat (spam)	Detectie prin hash content; blocat cu HTTP 409 dupa 3 postari identice in 5 minute	"Ai publicat deja acest continut recent."

3.6.4 Cazuri de utilizare administrative

- Moderare continut: moderatorul acceseaza panoul web de administrare, vizualizeaza rapoartele utilizatorilor, poate sterge postari sau suspenda conturi
- Export date GDPR: utilizatorul solicita exportul datelor; sistemul genereaza un arhiv ZIP cu toate datele in format JSON/CSV; link trimis pe email in maxim 24 ore
- Stergere cont (dreptul la uitare): contul este dezactivat imediat; datele personale sunt anonimizate in 30 de zile (postările raman cu [utilizator sters])
- Backup automat: baza de date este backupata zilnic in AWS S3 (retentie 30 zile); testare restore lunar

4. Planul de Implementare — 4 Sprint-uri

Proiectul Y Social Media este implementat in 4 sprint-uri Agile de cate 2 saptamani fiecare, pe o durata totala de 8 saptamani (Aprilie - Mai 2026). Fiecare sprint livreaza un increment functional si testabil.

Sprint 1 — Implementare 1: Fundatia & Autentificarea

SPRINT 1 | Saptamanile 1-2: 07 Apr - 20 Apr 2026 | Implementare 1			
Categorie	**Task**	**Responsabil**	**Status**
Setup & Infrastructura	Initializare repo GitHub, branch strategy (main/dev/feature/*), .gitignore, README	Toata echipa	Planificat
Setup & Infrastructura	Configurare proiect Node.js/Express: structura MVC, ESLint, Prettier, variabile de mediu (.env)	Backend dev	Planificat
Setup & Infrastructura	Configurare proiect React Native: navigare (React Navigation), structura componente, tema UI	Frontend dev	Planificat
Baza de date	Instalare si configurare PostgreSQL; creare schema initiala (tabela users, sessions); migrari	Backend dev	Planificat
Baza de date	Configurare Redis pentru cache sesiuni si blacklist JWT	Backend dev	Planificat
Autentificare Backend	POST /auth/register — validare date, hash parola bcrypt, creare cont, trimitere email confirmare	Backend dev	Planificat
Autentificare Backend	POST /auth/login — verificare credentiale, generare JWT access + refresh token	Backend dev	Planificat
Autentificare Backend	POST /auth/logout — invalidare token in Redis blacklist	Backend dev	Planificat
Autentificare Backend	POST /auth/refresh — reinoire access token cu refresh token valid	Backend dev	Planificat
Autentificare Backend	POST /auth/forgot-password — generare link reset parola, trimitere email (SendGrid)	Backend dev	Planificat
Profil de baza	GET /users/:id — returnare date profil public; GET /me — date profil propriu	Backend dev	Planificat
Frontend Auth	Ecran Sign Up: formular validat (email, username, parola), feedback erori, buton inregistrare	Frontend dev	Planificat
Frontend Auth	Ecran Sign In: autentificare, stocare	Frontend dev	Planificat

	token securizat (SecureStore), redirect la feed		
Frontend Auth	Ecran Recuperare Parola: introducere email, confirmare trimitere email	Frontend dev	Planificat
Testare Sprint 1	Unit tests AuthController (Jest); testare manuala fluxuri auth; validare securitate (parole hash, JWT)	Toata echipa	Planificat

Deliverable ADD acoperit	Criterii de acceptanta Sprint 1
§3.1 AUTH Subsystem, §3.3 Schema users, §3.4 Control acces RBAC, §3.6 Conditii limita auth	Utilizatorul poate crea cont, autentifica, deautentifica si recupera parola. JWT functional. Parole stocate hashed. Redis blacklist operational.

Sprint 2 — Implementare 2: Core Functionalities (Feed & Postari)

SPRINT 2 | Saptamanile 3-4: 21 Apr - 04 Mai 2026 | Implementare 2			
Categorie	**Task**	**Responsabil**	**Status**
Scheme BD Sprint 2	Creare tabele: posts, comments, likes, reposts, hashtags, post_hashtags cu toate relatiile si indexurile	Backend dev	Planificat
API Postari	POST /posts — creare postare (text + optional media), validare 280 chars, parsare hashtag-uri si mentionari	Backend dev	Planificat
API Postari	GET /posts/:id — detalii postare cu like_count, comment_count, repost_count, autorul	Backend dev	Planificat
API Postari	DELETE /posts/:id — stergere postare proprie (verificare proprietar prin JWT)	Backend dev	Planificat
API Media	POST /posts/:id/media — upload imagine/GIF la AWS S3 (Multer + Sharp compresie, max 5MB), return URL CDN	Backend dev	Planificat
API Feed	GET /feed — feed cronologic cu postarile utilizatorilor urmäriti; paginare cursor-based; cache Redis 5min	Backend dev	Planificat
API Interactiuni	POST /posts/:id/like si DELETE /posts/:id/like — toggle like, actualizare contor	Backend dev	Planificat
API Interactiuni	POST /posts/:id/comments — adaugare comentariu; GET /posts/:id/comments — lista comentarii cu paginare	Backend dev	Planificat
API Interactiuni	POST /posts/:id/repost si DELETE /posts/:id/repost — repostare, actualizare contor	Backend dev	Planificat
Profil extins	PATCH /me/profile — editare display_name, bio, avatar (upload S3); GET /users/:id/posts, /likes, /media	Backend dev	Planificat
Frontend Feed	Ecran principal (Home/Feed): lista postari cu card (avatar, username, text, media, like/comment/repost)	Frontend dev	Planificat
Frontend Feed	Modal creare postare: textarea cu contor 280 chars, picker imagine/GIF, buton Posteaza	Frontend dev	Planificat
Frontend Profil	Ecran profil utilizator: avatar, bio, statistici (postari/urmäritori/urmäriti), tab-uri Postari/Media/Like-uri	Frontend dev	Planificat

Frontend Detaliu Postare	Ecran detaliu postare: postare completa + sectiune comentarii + buton adauga comentariu	Frontend dev	Planificat
Testare Sprint 2	Unit tests PostController si FeedService; testare upload media; testare interactiuni (like/comment/repost)	Toata echipa	Planificat

Deliverable ADD acoperit	Criterii de acceptanta Sprint 2
§3.1 CONTENT Subsystem, §3.3 Schema BD (posts/comments/likes), §3.5 Flux global control (postare)	Utilizatorul poate publica postari cu text si media, vizualiza feed-ul, da like/comment/repost. Upload imagini functional. Feed cu cache Redis.

Sprint 3 — Implementare 3: Social Features, Mesagerie & Notificari

SPRINT 3 | Saptamanile 5-6: 05 Mai - 18 Mai 2026 | Implementare 3			
Categorie	**Task**	**Responsabil**	**Status**
Scheme BD Sprint 3	Creare tabele: followers, conversations, messages, read_receipts, notifications cu indexuri optimizate	Backend dev	Planificat
API Social - Follow	POST /users/:id/follow — follow/unfollow utilizator; suport conturi private (status pending/active)	Backend dev	Planificat
API Social - Follow	GET /users/:id/followers si /following — liste urmäritori/urmäriti cu paginare; PUT /follow-requests/:id	Backend dev	Planificat
API Cautare	GET /search?q=... — cautare full-text utilizatori (username, display_name), grupuri si hashtag-uri in timp real	Backend dev	Planificat
API Hashtag-uri	GET /hashtags/:tag/posts — postari dupa hashtag; GET /hashtags/trending — top 10 hashtag-uri (Redis Sorted Set)	Backend dev	Planificat
WebSocket Setup	Configurare Socket.io server cu autentificare JWT la conectare; Redis Pub/Sub pentru scalare multi-instanta	Backend dev	Planificat
API Mesagerie	GET /conversations — lista conversatii; POST /conversations — creare conversatie noua; GET /conversations/:id/messages	Backend dev	Planificat
Real-time Mesagerie	WebSocket: trimitere/primire mesaje in timp real (emit/on message); status citire (read receipt); reconectare automata	Backend dev	Planificat
API Notificari	GET /notifications — lista notificari; PATCH /notifications/:id/read; DELETE /notifications (stergere toate)	Backend dev	Planificat
Push Notifications	Integrare Firebase Admin SDK (FCM) pentru Android si APNs pentru iOS; trimitere push la like/comment/follow/DM	Backend dev	Planificat
Frontend Cautare	Ecran Cautare: bara search, rezultate live (utilizatori cu badge Online, hashtag-uri trending, grupuri)	Frontend dev	Planificat
Frontend Social	Buton Follow/Unfollow pe profil; feed filtrat dupa urmäriti; ecran lista urmäritori/urmäriti	Frontend dev	Planificat

Frontend Mesaje	Ecran Mesaje: lista conversatii (avatar, preview, timestamp); ecran chat cu bule mesaje si input text	Frontend dev	Planificat
Frontend Notificari	Ecran Notificari: lista cu icon tip (inima/comentariu/repost/follow), text descriptiv, thumbnail, badge tab	Frontend dev	Planificat
Testare Sprint 3	Testare WebSocket (conectare, deconectare, reconectare); testare push notifications pe device real; testare cautare	Toata echipa	Planificat

Deliverable ADD acoperit	Criterii de acceptanta Sprint 3
§3.1 SOCIAL + MESSAGING + NOTIFICATIONS, §3.2 Distributie WebSocket, §3.3 Schema BD completa	Follow/unfollow functional. Mesaje in timp real sub 500ms. Notificari push livrate pe iOS si Android. Cautare returneaza rezultate relevante.

Sprint 4 — Finalizare: Polish, Testare, Securitate & Deployment

SPRINT 4 | Saptamanile 7-8: 19 Mai - 01 Iun 2026 | Finalizare			
Categorie	**Task**	**Responsabil**	**Status**
UI/UX Polish	Implementare design final: gradient albastru-violet (#4FC3F7 -> #7C4DFF), animatii, dark mode	Frontend dev	Planificat
UI/UX Polish	Responsive design: testare pe ecrane 4.7" pana la 6.9"; accesibilitate (contrast, font size, screen reader)	Frontend dev	Planificat
UI/UX Polish	Onboarding flow (max 3 pasi): Bun venit, Completeaza profil, Urmareste utilizatori recomandat	Frontend dev	Planificat
Moderare & Raportare	POST /posts/:id/report si /users/:id/report — raportare cu categorii (spam/abuz/continut inadecvat)	Backend dev	Planificat
Moderare & Raportare	Panou de administrare web: vizualizare rapoarte, actiuni de moderare (stergere postare, suspendare cont)	Backend dev	Planificat
Securitate	Audit securitate: rate limiting (express-rate-limit), protectie XSS (helmet.js), validare input (joi/zod) pe toate endpoint-urile	Backend dev	Planificat
Securitate	Testare penetrare de baza: SQL injection, XSS, CSRF, brute-force login; corectare vulnerabilitati gasite	Toata echipa	Planificat
GDPR & Confidentialitate	POST /me/export-data — generare arhiv ZIP cu toate datele utilizatorului; DELETE /me — stergere cont (anonimizare)	Backend dev	Planificat
GDPR & Confidentialitate	Pagina Termeni si Conditii + Politica de Confidentialitate afisate la inregistrare si in setari cont	Frontend dev	Planificat
Testare & QA	Testare automata backend: minimum 70% code coverage (Jest); integration tests pentru toate fluxurile principale	Toata echipa	Planificat
Testare & QA	Load testing: simulare 1000 utilizatori concurenti (k6/Artillery); verificare p95 < 2s; optimizare bottleneck-uri	Backend dev	Planificat
Testare & QA	Testare manuala completa pe iOS (TestFlight) si Android (APK intern): toate fluxurile, edge cases, erori	Toata echipa	Planificat
CI/CD & Deployment	Configurare GitHub Actions: pipeline build, lint, test, deploy automat la	Backend dev	Planificat

	push pe main		
CI/CD & Deployment	Containerizare Docker + deploy backend pe AWS EC2/ECS; configurare RDS PostgreSQL + ElastiCache Redis	Backend dev	Planificat
CI/CD & Deployment	Build app mobila pentru testare: Expo EAS Build pentru iOS (TestFlight) si Android (Google Play Internal)	Frontend dev	Planificat
Documentatie	Documentatie API completa (Swagger/OpenAPI 3.0) pentru toate endpoint-urile; Postman collection exportata	Backend dev	Planificat
Documentatie	Finalizare ADD (documentul curent), README complet cu instructiuni de instalare si rulare locala	Toata echipa	Planificat

Deliverable ADD acoperit	Criterii de acceptanta Sprint 4
§3.6 Conditii limita, §3.2 Distributie cloud, Glosar de termeni, Documentatie API completa	Aplicatia functionala pe iOS si Android. Load test trecut. Audit securitate fara vulnerabilitati critice. API documentat. CI/CD pipeline operational. GDPR implementat.

5. Calendarul General al Proiectului

Sprint	Perioada	Obiectiv principal	Milestone
Sprint 1	07 Apr - 20 Apr 2026	Implementare 1: Fundatie & Autentificare	Cont creat, autentificare JWT functionala
Sprint 2	21 Apr - 04 Mai 2026	Implementare 2: Feed, Postari & Interactiuni	Feed functional cu postari, like, comentariu, repost
Sprint 3	05 Mai - 18 Mai 2026	Implementare 3: Social, Mesagerie & Notificari	Follow, chat real-time, notificari push livrate
Sprint 4	19 Mai - 01 Iun 2026	Finalizare: Polish, Testare & Deployment	App live pe TestFlight/Play Internal, documentatie completa

6. Schema Bazei de Date (ERD)

Sistemul Y utilizeaza PostgreSQL 15 ca baza de date relationala. Diagrama de mai jos prezinta toate tabelele, campurile, tipurile de date, constrangerile si relatiile dintre entitati.

6.1 Diagrama Relationala (ERD) — Tabele si Relatii

TABELA: users			
Coloana	**Tip date**	**Constrangeri**	**Descriere**
id	UUID DEFAULT gen_random_uuid()	**PK, NOT NULL**	Cheie primara
username	VARCHAR(50)	UNIQUE, NOT NULL	Nume unic utilizator
email	VARCHAR(255)	UNIQUE, NOT NULL	Email adresa
password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt parola
display_name	VARCHAR(100)	NULL	Nume afisat profil
bio	TEXT	NULL	Descriere profil
avatar_url	VARCHAR(500)	NULL	URL poza profil (S3)
role	ENUM('user','moderator','admin') DEFAULT 'user'	NOT NULL	Rol RBAC
is_private	BOOLEAN DEFAULT FALSE	NOT NULL	Cont privat sau public
is_verified	BOOLEAN DEFAULT FALSE	NOT NULL	Cont verificat (badge)
email_verified_at	TIMESTAMP WITH TIME ZONE	NULL	Data confirmare email
created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()	NOT NULL	Data creare cont
updated_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()	NOT NULL	Data ultima modificare
INDEX: idx_users_username (username), idx_users_email (email) **RELATII: 1-N posts, 1-N notifications, 1-N conversations; N-M followers (self-referential)**			

TABELA: posts			
Coloana	**Tip date**	**Constrangeri**	**Descriere**
id	UUID DEFAULT gen_random_uuid()	**PK, NOT NULL**	Cheie primara
user_id	UUID	FK -> users.id	Autorul postarii

content	VARCHAR(280)	NOT NULL	Textul postarii
media_url	VARCHAR(500)	NULL	URL CDN imagine/GIF
media_type	ENUM('image','gif','video')	NULL	Tip fisier media
parent_id	UUID	FK -> posts.id, NULL	Postare parinte (reply)
like_count	INTEGER DEFAULT 0	NOT NULL, >= 0	Contor like-uri
comment_count	INTEGER DEFAULT 0	NOT NULL, >= 0	Contor comentarii
repost_count	INTEGER DEFAULT 0	NOT NULL, >= 0	Contor repostari
created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()	NOT NULL	Data publicarii
INDEX: idx_posts_user_id (user_id), idx_posts_created_at (created_at DESC), idx_posts_parent_id (parent_id) **RELATII: N-1 users; 1-N likes, comments, reposts; N-M hashtags; self-referential parent_id**			

6.2 Tabelele de Relatie si Interactiune

TABELA: followers (relatie N-M self-referentiala users)			
Coloana	**Tip date**	**Constrangeri**	**Descriere**
follower_id	UUID	**PK, FK -> users.id**	Cel care urmareste
following_id	UUID	**PK, FK -> users.id**	Cel urmarit
status	ENUM('active','pending') DEFAULT 'active'	NOT NULL	Activ / In asteptare
created_at	TIMESTAMP WITH TIME ZONE DEFAULT NOW()	NOT NULL	Data urmaririi
PK COMPOZITA: (follower_id, following_id) | CONSTRAINT: follower_id != following_id **INDEX: idx_followers_following_id (following_id) pentru feed rapid**			

TABELA: messages (si conversations)			
Coloana	**Tip date**	**Constrangeri**	**Tabela**
conversations.id	UUID DEFAULT gen_random_uuid()	**PK, NOT NULL**	conversations
conversations.participant1_id	UUID	FK -> users.id	conversations
conversations.participant2_id	UUID	FK -> users.id	conversations

messages.conversation_id	UUID	FK -> conversations.id	messages
messages.content	TEXT	NOT NULL	messages
messages.is_read	BOOLEAN DEFAULT FALSE	NOT NULL	messages
INDEX: idx_messages_conversation_id, idx_conversations_participants (participant1_id, participant2_id UNIQUE)			

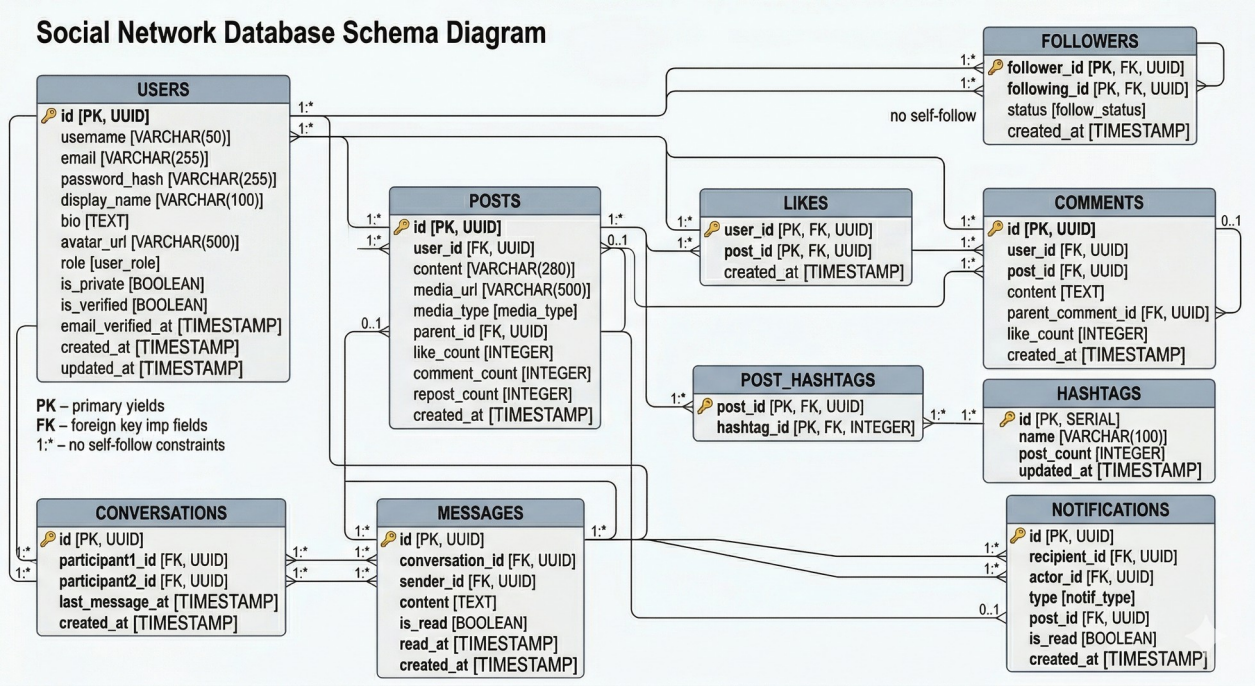
6.3 Script SQL de Creare a Bazei de Date

Urmatorul script SQL creeaza schema completa a bazei de date PostgreSQL pentru proiectul Y:

```
-- ============================================
-- Y Social Media — PostgreSQL 15 Schema
-- ============================================

CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ENUM types
CREATE TYPE user_role AS ENUM ('user', 'moderator', 'admin');
CREATE TYPE follow_status AS ENUM ('active', 'pending');
CREATE TYPE media_type AS ENUM ('image', 'gif', 'video');
CREATE TYPE notif_type AS ENUM ('like', 'comment', 'repost', 'follow', 'mention');

-- TABLE: users
CREATE TABLE users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username          VARCHAR(50)  UNIQUE NOT NULL,
  email             VARCHAR(255) UNIQUE NOT NULL,
  password_hash     VARCHAR(255) NOT NULL,
  display_name      VARCHAR(100),
  bio               TEXT,
  avatar_url        VARCHAR(500),
  role              user_role    NOT NULL DEFAULT 'user',
  is_private        BOOLEAN      NOT NULL DEFAULT FALSE,
  is_verified       BOOLEAN      NOT NULL DEFAULT FALSE,
  email_verified_at TIMESTAMPTZ,
  created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_email    ON users(email);

-- TABLE: posts
CREATE TABLE posts (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  content       VARCHAR(280) NOT NULL,
  media_url     VARCHAR(500),
  media_type    media_type,
  parent_id     UUID         REFERENCES posts(id) ON DELETE SET NULL,
  like_count    INTEGER      NOT NULL DEFAULT 0 CHECK (like_count >= 0),
  comment_count INTEGER      NOT NULL DEFAULT 0 CHECK (comment_count >= 0),
  repost_count  INTEGER      NOT NULL DEFAULT 0 CHECK (repost_count >= 0),
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_posts_user_id    ON posts(user_id);
CREATE INDEX idx_posts_created_at ON posts(created_at DESC);
```

```sql
-- TABLE: followers
CREATE TABLE followers (
  follower_id  UUID         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  following_id UUID         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  status       follow_status NOT NULL DEFAULT 'active',
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
  PRIMARY KEY (follower_id, following_id),
  CONSTRAINT no_self_follow CHECK (follower_id != following_id)
);
CREATE INDEX idx_followers_following_id ON followers(following_id);

-- TABLE: likes (PK compozit)
CREATE TABLE likes (
  user_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  post_id    UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, post_id)
);

-- TABLE: comments
CREATE TABLE comments (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID    NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  post_id           UUID    NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  content           TEXT    NOT NULL,
  parent_comment_id UUID    REFERENCES comments(id) ON DELETE CASCADE,
  like_count        INTEGER NOT NULL DEFAULT 0 CHECK (like_count >= 0),
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- TABLE: hashtags + post_hashtags
CREATE TABLE hashtags (
  id         SERIAL PRIMARY KEY,
  name       VARCHAR(100) UNIQUE NOT NULL,
  post_count INTEGER NOT NULL DEFAULT 0,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE post_hashtags (
  post_id    UUID    NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  hashtag_id INTEGER NOT NULL REFERENCES hashtags(id) ON DELETE CASCADE,
  PRIMARY KEY (post_id, hashtag_id)
);

-- TABLE: conversations + messages
CREATE TABLE conversations (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  participant1_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  participant2_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  last_message_at TIMESTAMPTZ,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (participant1_id, participant2_id)
);
CREATE TABLE messages (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id  UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
  sender_id        UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  content          TEXT NOT NULL,
  is_read          BOOLEAN NOT NULL DEFAULT FALSE,
  read_at          TIMESTAMPTZ,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_messages_conv_id ON messages(conversation_id, created_at DESC);

-- TABLE: notifications
CREATE TABLE notifications (
```

```
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  recipient_id UUID         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  actor_id     UUID         NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type         notif_type   NOT NULL,
  post_id      UUID         REFERENCES posts(id) ON DELETE CASCADE,
  is_read      BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_notif_recipient ON notifications(recipient_id, created_at DESC);
CREATE INDEX idx_notif_unread   ON notifications(recipient_id) WHERE is_read =
FALSE;
```



Glosar de Termeni

Termen	Definitie extins
Micro-blogging	Forma de blogging care permite publicarea de continut scurt (text, imagini) distribuit rapid in retea.
REST API	Interfata de programare bazata pe principii RESTful: stateless, resurse identificate prin URL, operatii prin metode HTTP (GET/POST/PATCH/DELETE).
JWT (JSON Web Token)	Standard open (RFC 7519) pentru transmiterea sigura de informatii intre parti ca obiect JSON semnat. Folosit pentru autentificare stateless.
bcrypt	Algoritm de hashing adaptiv pentru parole, cu factor de cost configurabil. Protectie impotriva atacurilor brute-force.
WebSocket	Protocol de comunicare full-duplex peste o singura conexiune TCP. Permite comunicare bidirectionala in timp real intre client si server.
Redis Pub/Sub	Mecanism de publicare-abonare din Redis folosit pentru distribuirea mesajelor in timp real intre mai multe instante de server.
CDN (Content Delivery Network)	Retea distribuita de servere care livreaza continut (imagini, video) din locatii geografice apropiate de utilizator, reducand latenta.
Cursor-based pagination	Tehnica de paginare bazata pe un cursor (ID sau timestamp) in loc de offset numeric. Mai eficienta pentru seturi mari de date in continua schimbare.
Graceful Shutdown	Oprirea controlata a unui server care finalizeaza cererile active inainte de a se opri, fara pierdere de date.
RBAC (Role-Based Access Control)	Model de control al accesului in care permisiunile sunt asociate cu roluri (Guest, User, Moderator, Admin), nu individual cu fiecare utilizator.
Exponential Backoff	Strategie de reconectare in care intervalul dintre reincercari creste exponential (1s, 2s, 4s, ...) pentru a reduce supraincarcarea serverului.
CI/CD	Continuous Integration/Continuous Deployment: automatizarea proceselor de build, testare si deploy al aplicatiei la fiecare modificare de cod.
GDPR	General Data Protection Regulation: regulamentul european (2016/679) privind protectia datelor personale ale cetatenilor UE.